DIGITAL SIGNAL PROCESSING

Under Supervision of:

Dr. Michael Ibrahim
Eng. Ahmed Khaled

Submitted by:

Ahmed Belal	**23P0007**
Ahmed Sherif	**23P0404**
Youssef Wael	**2300047**
Mayar Hossam	**2300445**

Table of Contents

INTRODUCTION 3
PLOT FILTER ANALYSIS FUNCTION 4
MEAN SQUARE CALCULATION 6
MEAN POWER 7
TASK 0: Audio File Analysis 8
TASK 1: Digital Echo System 10
TASK 2: Design of a Digital LPF 15
TASK 3: Frequency Transformation using Pole-Zero Pattern Rotation 19

INTRODUCTION

Digital Signal Processing (DSP) plays a fundamental role in modern communication systems, audio processing, and real-time signal enhancement. In many practical applications, signals undergo distortion due to channel effects, noise, echoes, and unwanted frequency components. The aim of this project is to design, analyze, and implement several digital systems and filters using MATLAB in order to understand their behavior in both the time and frequency domains.

This project begins with analyzing a real audio signal and computing its time-domain characteristics, spectrum, and energy. Then, a digital echo system is implemented, and its impact on the audio signal is studied through impulse response, frequency response, pole–zero analysis and mean-square-error (MSE) evaluation. To compensate for echo distortion, an equalizer system is derived analytically and implemented digitally, demonstrating how inverse filtering restores the original signal.

The project continues with the design of different types of digital IIR low-pass filters—including Butterworth, Chebyshev Type I, Chebyshev Type II, and Elliptic filters—to meet given specifications. Each filter is analyzed, compared, and applied to the audio signal to evaluate performance in terms of MSE and energy loss. Finally, frequency transformation techniques are applied to convert a low-pass filter into high-pass and band-pass filters by rotating the pole-zero pattern.

Throughout the project, MATLAB is used to generate pole-zero diagrams, magnitude and phase responses, group delay plots, impulse responses, and filtered audio outputs. The results illustrate the practical behavior of digital filters and the effect of system characteristics on real signals.

PLOT FILTER ANALYSIS FUNCTION

This function is used to generate all required plots for any digital filter.
When we give it the filter coefficients (b and a), the sampling frequency, delay, and number of taps, it creates one figure that contains:

- Pole–zero plot
- Magnitude response
- Phase response
- Group delay
- Impulse response

This saves time because we use these plots many times in the project, so instead of rewriting the same code, we just call this function.

Code:

```
function  plot_filter_analysis(b , a , Fs, D, k)
% b --> Numerator
% a --> Denominator
% Fs --> Sampling Frequency
% D --> Delay value
% k --> Number of coefficients

%% Create one figure window for all plots
figure('Position', [50, 50, 1400, 700]);

%% Ploting Pole-Zero
subplot(3,2,1) ;
zplane(b,a) ;
title('Pole-Zero Plot') ;
grid on ;

%% Frequency Response Calculation
frmsz = 8192 ; % 2^13
f = (-frmsz/2:(frmsz/2)-1)*(Fs/frmsz) ; % Frequency axis
[H, Wf] = freqz(b , a , frmsz , "whole") ; % Frequency response
% Without 'whole' You get 0 to pi ,but With 'whole' You get 0 to 2*pi
% So we use it because he said get it for -Fs/2 to Fs/2
H_shift = fftshift(H) ;
% moves zero-frequency to the center of the spectrum.
% |___________|___________| ---> |_____|_____0_____|_____|
% 0           Fs            --->   -Fs/2          Fs/2
```

```
%% Magnitude Response
subplot(3,2,2) ;
plot(f, 20*log10(abs(H_shift))) ;
grid on ;
title('Magnitude Response in dB') ;
xlabel('Frequency (Hz)') ;
ylabel('Magnitude (dB)') ;

%% Phase Response
subplot(3,2,3) ;
plot(f, angle(H_shift)) ;
grid on ;
title('Phase Response') ;
xlabel('Frequency (Hz)') ;
ylabel('Phase (radians)') ;

%% Group Delay
[gd,Wg] = grpdelay (b,a,frmsz,'whole') ;
gd_shift = fftshift(gd) ;
subplot(3,2,4) ;
plot (f,gd_shift) ;
grid on ;
title('Group Delay') ;
xlabel('Frequency (Hz)') ;
ylabel('Group Delay (samples)') ;

%% Ploting Impulse Response
if D == 0
    % For regular filters (no echo), use enough samples to see decay
    n_samples = 100;  % Good for IIR filters
    delta = [1, zeros(1, n_samples-1)];
else
    % For echo systems, use the echo pattern length
    delta = [1, zeros(1, (k-1)*D)];
end
subplot(3,2,5) ;
imp_resp = filter(b, a, delta);
stem(0:length(imp_resp)-1, imp_resp, 'filled');
title('Impulse Response');
xlabel('Samples (n)');
ylabel('Amplitude');

end
```

MEAN SQUARE CALCULATION

This function calculates how much the filter changes the audio signal. It filters the input signal using **filter(b,a,yt)** and then computes the mean-square error (MSE) between the original and the filtered signal. It also returns the filtered signal so we can use it later. We use this function in many tasks (echo system, equalizer, LPF filters), so having it once makes the code cleaner.

Code:

```
function [mse_value,yf] = compute_mse(b , a , yt)
% b --> Numerator
% a --> Denominator
% yt --> Input signal
%
% Returns:
% mse_value --> Mean Square Error
% yf        --> Filtered output

%% Mean Square Calculation
yt = mean(yt,2) ; % stereo --> mono

% yf = conv(yt, h);
% This command is what you think you will do as you will apply
% convolution on the input with the impulse responce to get the output ,but
% bec. Convolution results in a longer signal (Length(x) + Length(h) - 1)
% so when subtracting the y1-yt the sizes won't be the same.

yf = filter(b, a, yt);
% Right Command bec. command is designed for systems where input
% and output must stay synchronized.

error_signal = yf - yt ; % Calculate the difference (error)
squared_error = error_signal .^ 2 ; % Square the error
mse_value = mean(squared_error) ; % Calculate the average (Mean)

disp(['The Mean Square Error is: ', num2str(mse_value)]); % Display the result

end
```

MEAN POWER

This function filters an input signal through a digital filter and computes the percentage of power lost due to filtering. It calls compute_mse() to obtain the filtered output, then calculates input and output power to determine the loss percentage.

Code:

```
% Inputs: b  : Numerator coefficients of the filter
%         a  : Denominator coefficients of the filter
%         yt : Input signal (can be stereo or mono)
%         filter_name : Filter name (string)
% Output: Displays the power loss percentage to the console
% Calls: compute_mse(b, a, yt) - Returns MSE and filtered output yf
function compute_power(b, a, yt, filter_name)
    [~, yf] = compute_mse(b, a, yt);
    pin = sum(yt.^2);
    pout = sum(yf.^2);
    loss_percentage = (pin-pout) / pin * 100;
    fprintf('Power Loss Percentage for %s is: %.2f%%\n\n', filter_name,
loss_percentage);
end
```

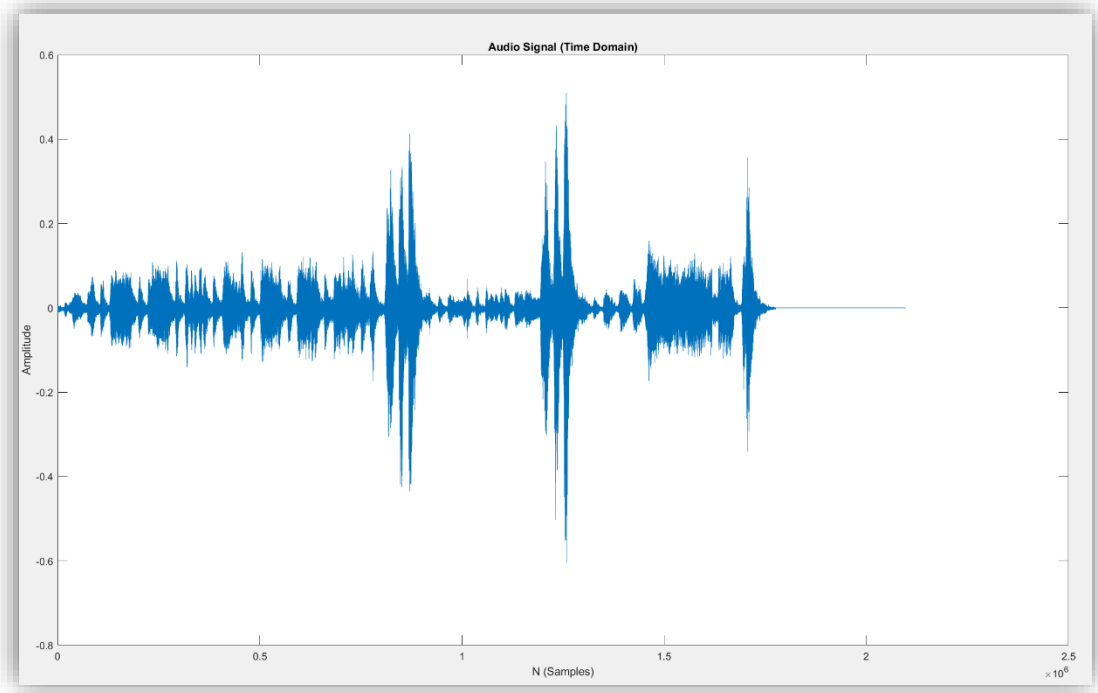
TASK 0: Audio File Analysis

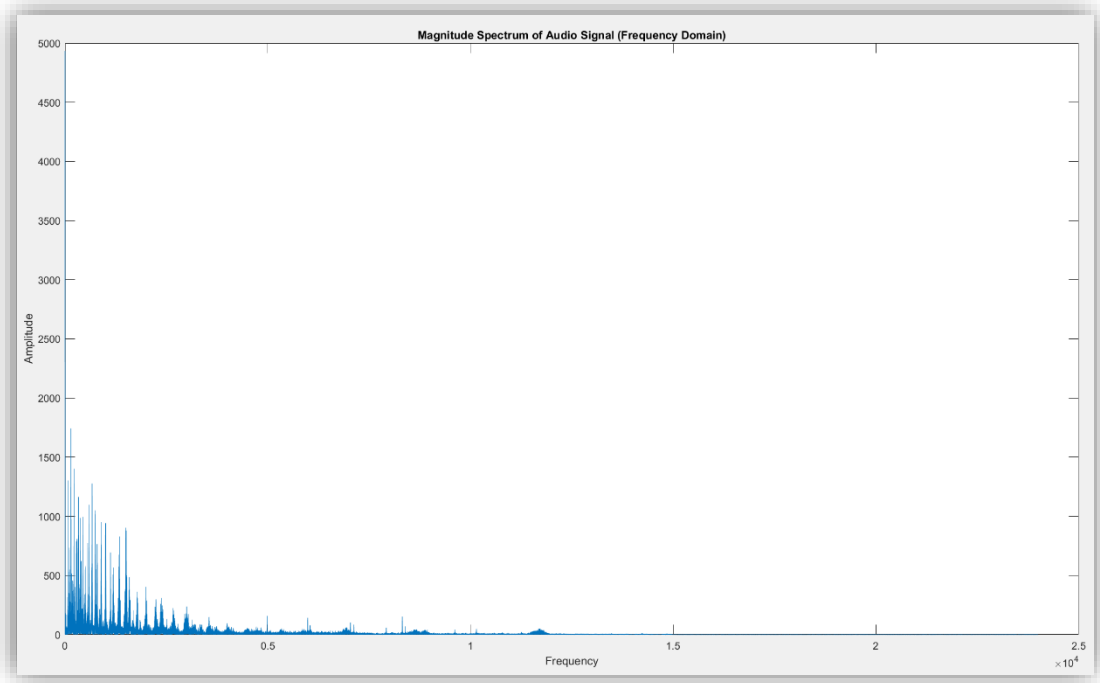
In this task, the provided audio file is analyzed to establish a baseline understanding of its characteristics. The audio is first imported and converted to a mono signal. Its time-domain waveform is plotted to visualize amplitude variations over time. The magnitude spectrum is then computed using the Fast Fourier Transform (FFT) to observe the distribution of frequency components. The total signal energy is calculated both in the time domain and the frequency domain to verify Parseval's theorem. This task provides the reference signal characteristics used throughout the rest of the project.

Code:

```
clear all;
close all;
clc;

%Read Audio File
[yt,Fs] = audioread("sample_audio_file.wav") ;


%Stereo --> Mono & Ploting Time Domain
yt=mean(yt,2) ; %due to that the audio signal is stereo so we got the mean to
make it mono
N = length(yt) ; %to get number of samples
plot (0:N-1,yt) ; %Plot time domain
title('Audio Signal (Time Domain)') ;
xlabel('N (Samples)') ;
ylabel('Amplitude') ;


%Frequency domain analysis & Ploting the Magnitude Spectrum
yf = abs(fft(yt)) ; % Magnitude of FFT of the signal (using abs function)
f = (0:(N-1)/2)*(Fs/N) ; %Sampling the frequency axis
figure
plot(f,yf(1:N/2));
title('Magnitude Spectrum of Audio Signal (Frequency Domain)') ;
xlabel('Frequency') ;
ylabel('Amplitude') ;


%Energy of Time Domain using Parseval's Relation
E = sum(yt.^2) ;
fprintf('Energy of Signal in Time Domain = %E\n',E) ;

%Energy of Frequency Domain using Parseval's Relation
E = (1/N)*sum(yf.^2) ;
fprintf('Energy of Signal in Frequncy Domain = %E\n',E) ;
```

Plots:





Energy Results:

```
Energy of Signal in Time Domain = 2.018626E+03
Energy of Signal in Frequncy Domain = 2.018626E+03
```

TASK 1: Digital Echo System

The purpose of this task is to model and analyze a digital echo system characterized by delayed and scaled copies of the input signal. The impulse response is constructed using delayed impulses based on the coefficients b_k and delay D. Hand analysis is used to derive the system's LCCDE and system function $H(z)$. MATLAB is then used to generate the pole-zero plot, magnitude response, phase response, group delay, and impulse response of the echo filter using the reusable analysis function.

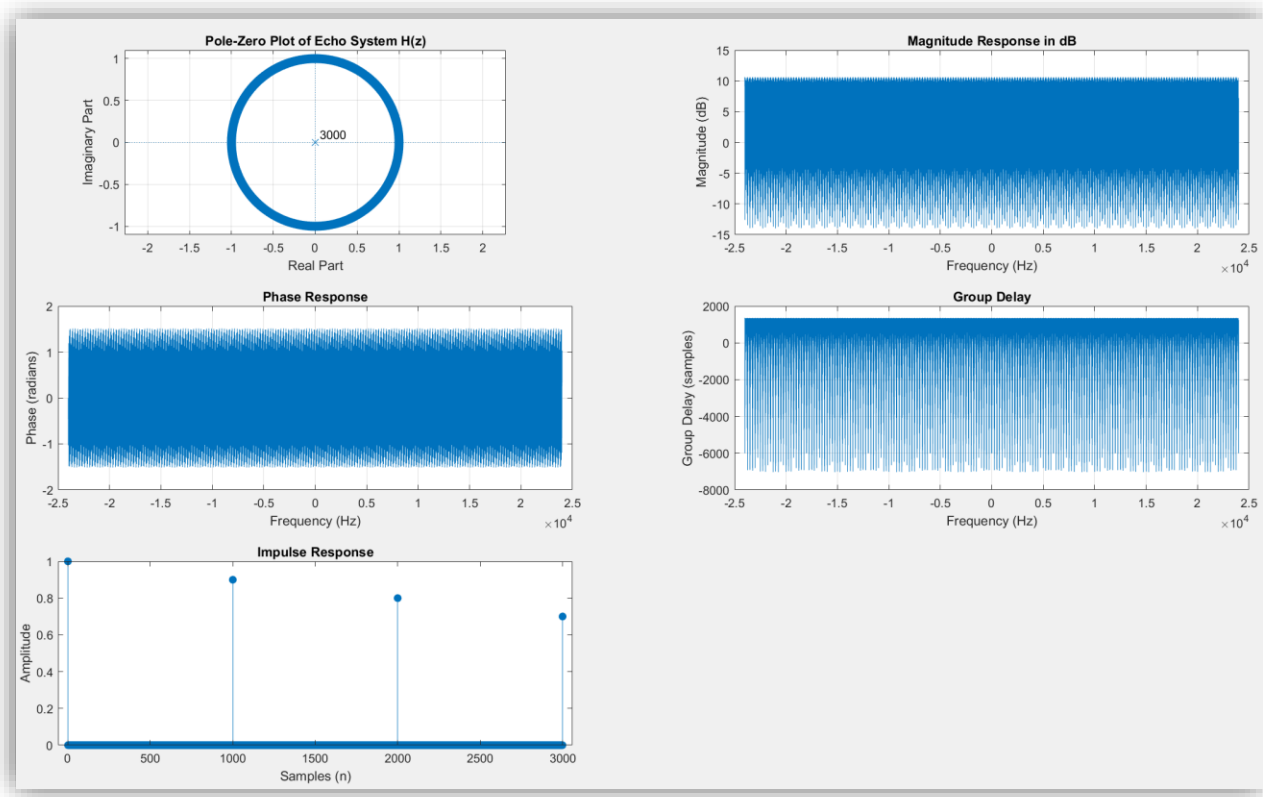
The echo system is applied to the audio signal, producing an echo-distorted output. The mean-square error (MSE) between the original and echo-affected signals is computed to quantify the amount of distortion introduced by the echo system.

Code:

```
close all;
clear all;
clc;

%Parameters
D = 1000 ;
bk = [1 , 0.9 , 0.8 , 0.7] ; % Impulse at 0,1000,2000,3000

%Read Audio File
[yt,Fs] = audioread("sample_audio_file.wav") ;


%% Digital Echo System H
k = length(bk) ; % Here K=4

% This inserts (D-1) zeros between every sample in bk
h = upsample(bk, D);
% Note: This makes the vector slightly longer (length 4000 instead of 3001)
% because it puts zeros after the last number too.
% Mathematically, this changes NOTHING. The filter is identical.

% Filter Coeffiecents
bh = h ;
ah = 1 ;
plot_filter_analysis(bh , ah , Fs , D ,k) ;

% Mean Square Calculation
[mse_h, y_h] = compute_mse(bh , ah , yt) ;
```

```
%% Equalizer System G.

% since ,G(Z) = 1/H(Z)
% Therefore ,the coefficients are reversed

% Filter Coeffiecents
ag = bh ;
bg = ah ;
plot_filter_analysis(bg , ag , Fs , D ,k) ;

% Mean Square Calculation
[mse_g, y_g] = compute_mse(bg , ag , yt) ;
```

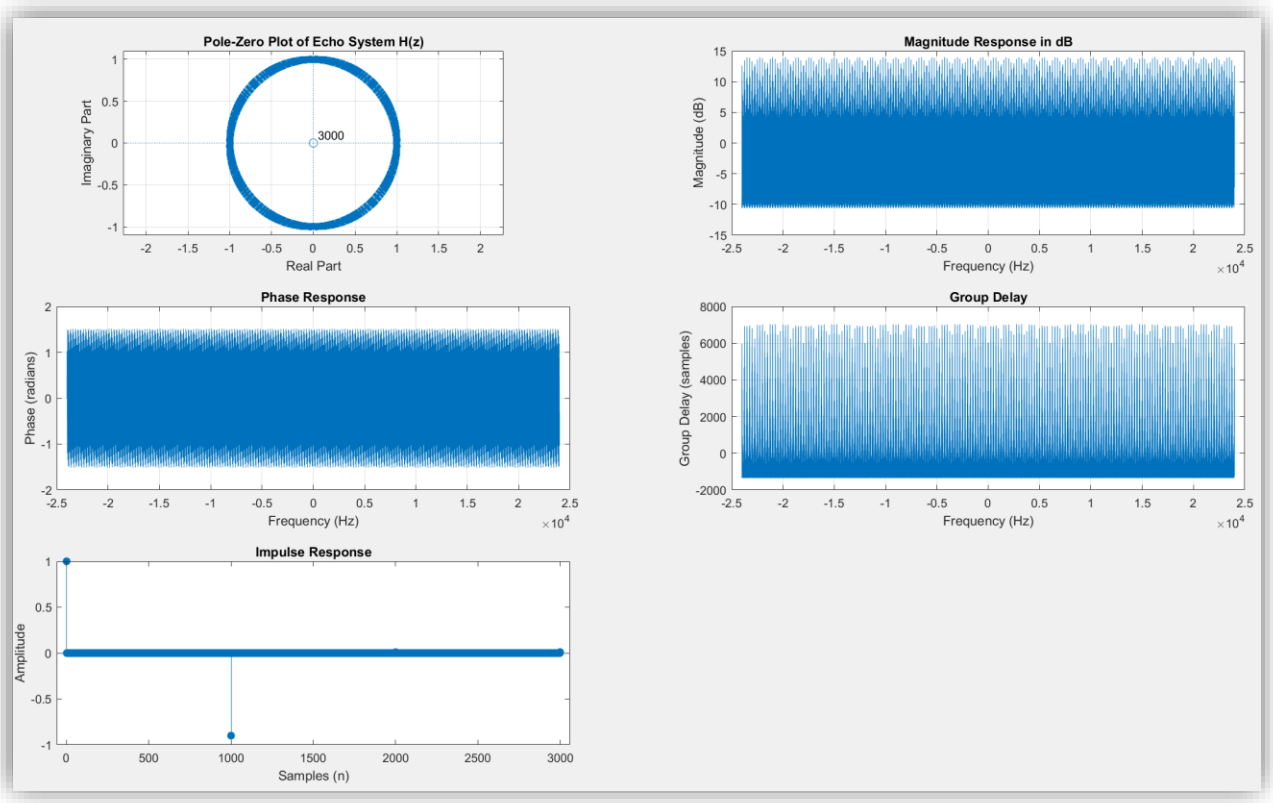
Plot of H(Z):



MSE of H(Z):

```
The Mean Square Error is: 0.0018882
```

Plot of G(Z):



MSE of G(Z):

```
The Mean Square Error is: 0.0020071
```

Hand Analysis:

Task 1

$$h[n] = \sum_{k=0}^{\infty} b_k \delta[n - kD]$$

- Let $b_k = \{\underline{1}, 0.9, 0.8, 0.7\}$, and let $D = 1000$.

☆ Digital Echo system:

$$h[n] = b_0 \delta[n-0] + b_1 \delta[n-D] + b_2 \delta[n-2D] + b_3 \delta[n-3D]$$
$$= 1\,\delta[n] + 0.9\,\delta[n-1000] + 0.8\,\delta[n-2000] + 0.7\,\delta[n-3000]$$

$$H(z) = 1 + 0.9\,z^{-1000} + 0.8\,z^{-2000} + 0.7\,z^{-3000}$$

get numerator and denominator to draw pole-zero plot

$b = [1, 0.9, 0.8, 0.7]$

$a = [1]$

$$\because H(z) = \frac{Y(z)}{X(z)}$$

$$Y(z) = H(z)\,X(z)$$
$$= X(z) + 0.9\,z^{-1000} X(z) + 0.8\,z^{-2000} X(z) + 0.7\,z^{-3000} X(z)$$

$$y[n] = x[n] + 0.9\,x[n-1000] + 0.8\,x[n-2000] + 0.7\,x[n-3000]$$

☆ Equalizer system:

$$H(Z)\,G(z) = 1$$

$$\therefore G(Z) = \frac{1}{H(Z)}$$

$$\therefore G(z) = \frac{1}{1 + 0.9\,Z^{-1000} + 0.8\,Z^{-2000} + 0.7\,Z^{-3000}}$$

$\therefore$ $b = [1]$

$a = [1, 0.9, 0.8, 0.7]$

$$G(z) = \frac{Y(z)}{X(z)}$$

$$\therefore Y(z)[1 + 0.9\,Z^{-1000} + 0.8\,Z^{-2000} + 0.7\,Z^{-3000}] = X(z)$$

$$\therefore y[n] + 0.9\,y[n-1000] + 0.8\,y[n-2000] + 0.7\,y[n-3000] = X[n]$$

$$\therefore y[n] = X[n] + 0.9\,y[n-1000] + 0.8\,y[n-2000] + 0.7\,y[n-3000]$$

TASK 2: Design of a Digital LPF

This task involves designing four minimum-order IIR low-pass filters that meet the specifications:

$$f_p = 3\text{kHz}, f_s = 4\text{kHz}, A_p \leq 1\text{dB}, A_s \geq 50\text{dB}$$

The filters designed are:

- Butterworth
- Chebyshev Type I
- Chebyshev Type II
- Elliptic

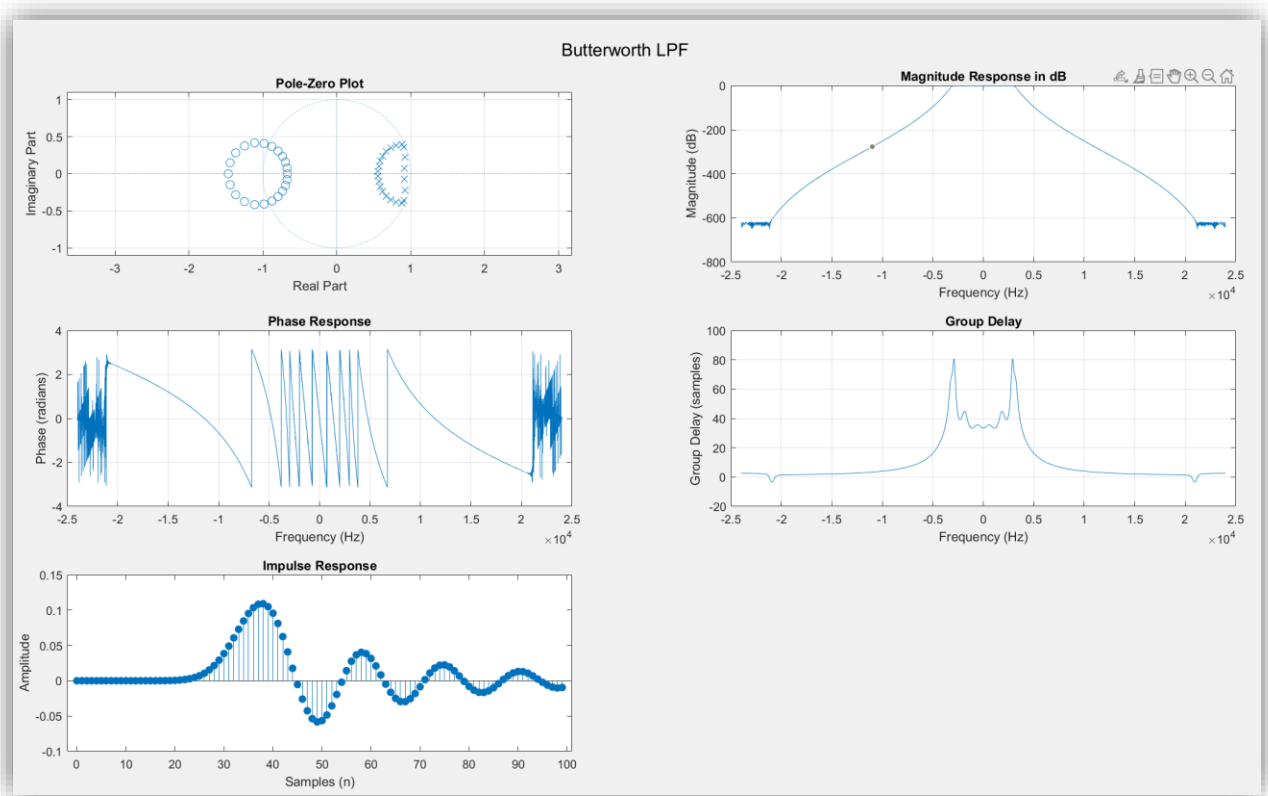
For each filter, MATLAB's filter design functions are used to determine the minimum filter order and coefficients. Each filter is analyzed using the reusable plotting function. The audio signal is passed through each filter, and both MSE and energy loss are computed. A comparison table is created to evaluate which filter provides the least distortion and which preserves the most signal energy. This task highlights the trade-offs among the four classical IIR filter types.

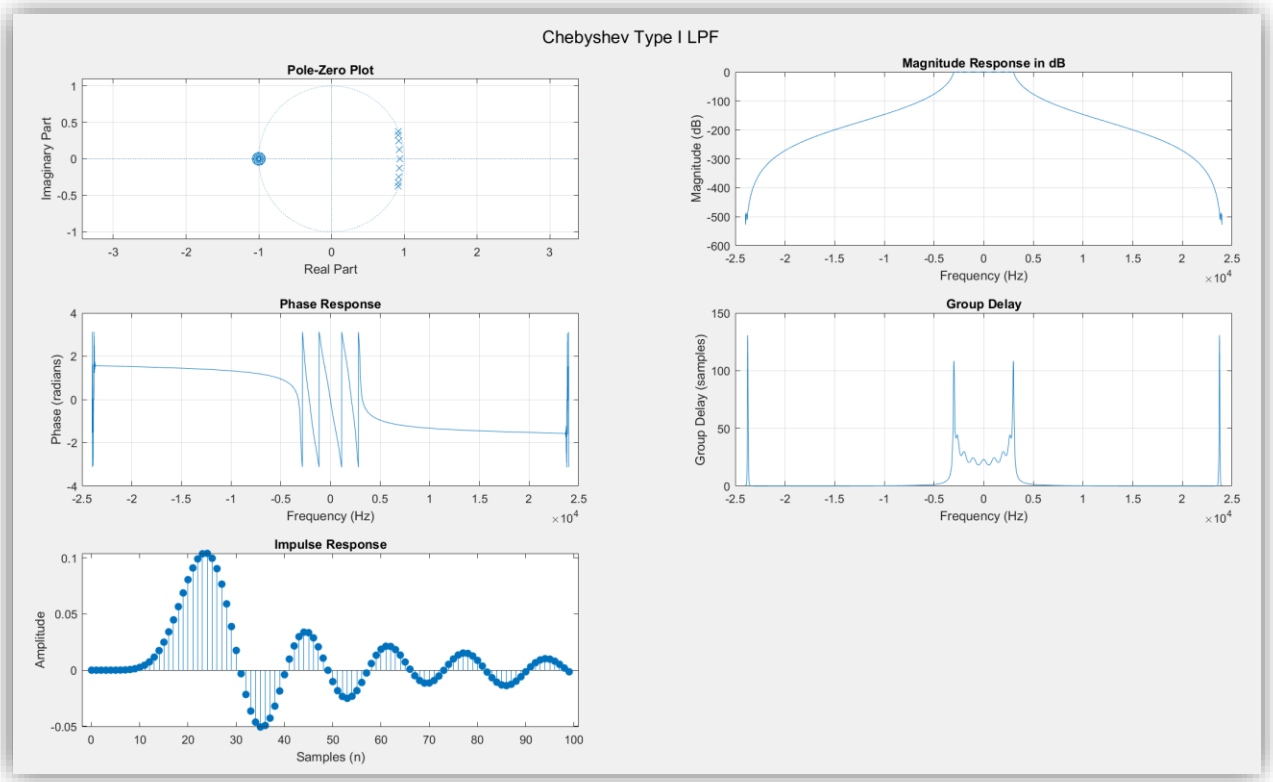
Code:

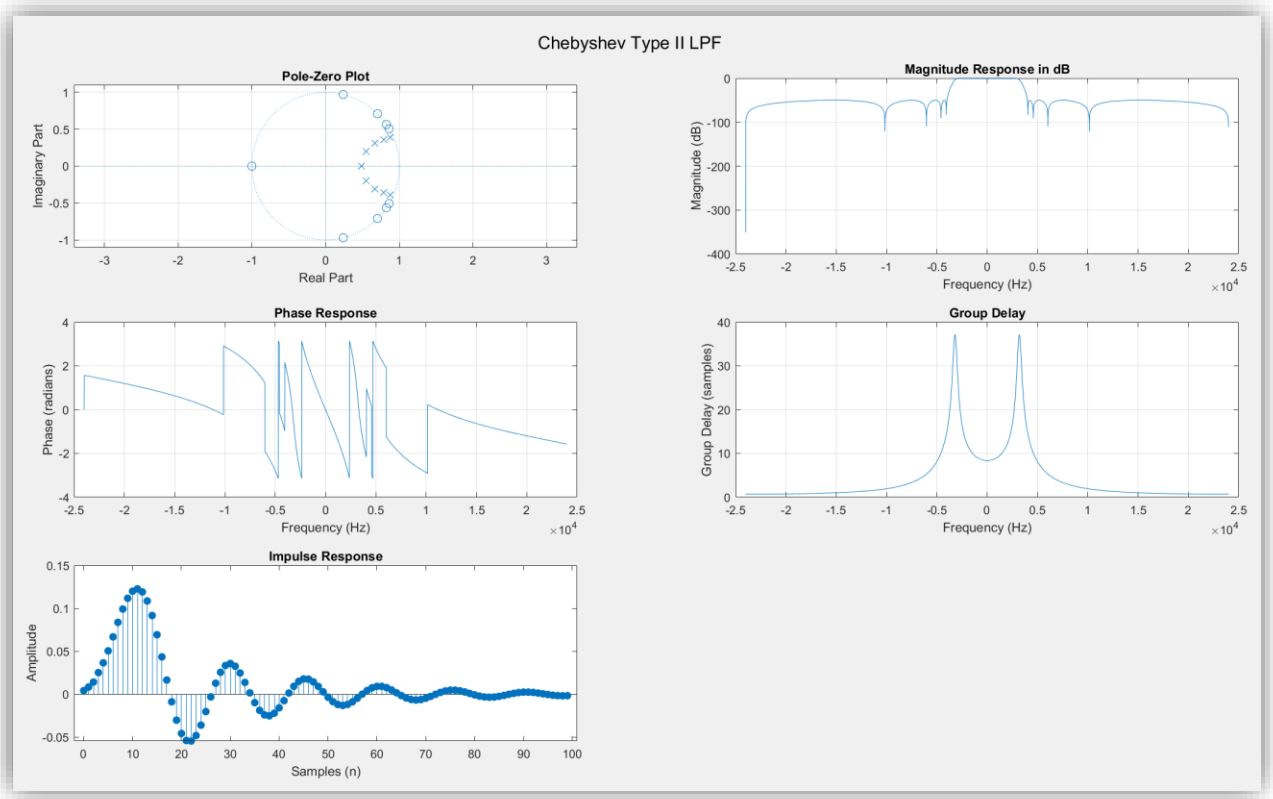
```
close all ;
clear all ;
clc ;

[x, Fs] = audioread('sample_audio_file.wav');
x = x(:,1);
Fp = 3000;
Fs_stop = 4000;
Ap = 1;
As = 50;
Wp = Fp / (Fs/2);
Ws = Fs_stop / (Fs/2);

%% Butterworth
[Nb, WnB] = buttord(Wp, Ws, Ap, As);
[bB, aB] = butter(Nb, WnB, 'low');
plot_filter_analysis(bB , aB , Fs, 0, 0);
sgtitle('Butterworth LPF');
compute_power(bB, aB, x, "Butterworth LPF")
```

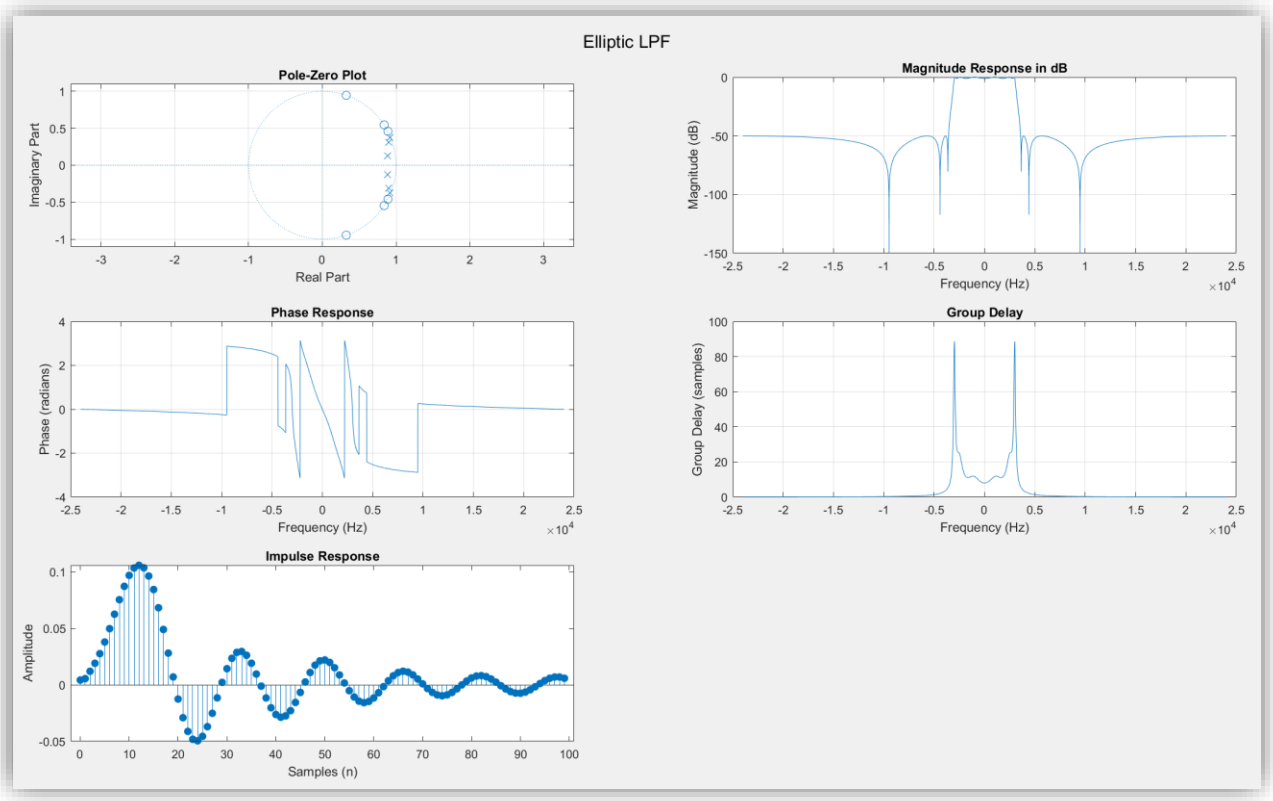
```
%% Chebyshev Type I
[Nc1, WpC1] = cheb1ord(Wp, Ws, Ap, As);
[bC1, aC1] = cheby1(Nc1, Ap, WpC1, 'low');
plot_filter_analysis(bC1 , aC1 , Fs, 0, 0);
sgtitle('Chebyshev Type I LPF');
compute_power(bC1, aC1, x, "Chebyshev Type I LPF")

%% Chebyshev Type II
[Nc2, WpC2] = cheb2ord(Wp, Ws, Ap, As);
[bC2, aC2] = cheby2(Nc2, As, WpC2, 'low');
plot_filter_analysis(bC2 , aC2 , Fs, 0, 0);
sgtitle('Chebyshev Type II LPF');
compute_power(bC2, aC2, x, "Chebyshev Type II LPF")

%% Elliptic
[Nc3, WpC3] = ellipord(Wp, Ws, Ap, As);
[bC3, aC3] = ellip(Nc3, Ap, As, WpC3, 'low');
plot_filter_analysis(bC3 , aC3 , Fs, 0, 0);
sgtitle('Elliptic LPF');
compute_power(bC3, aC3, x, "Elliptic LPF")
```

Plot:




Chebyshev Type I LPF
Pole-Zero Plot
Imaginary Part
Real Part
Magnitude Response in dB
Magnitude (dB)
Frequency (Hz)
Phase Response
Phase (radians)
Frequency (Hz)
Group Delay
Group Delay (samples)
Frequency (Hz)
Impulse Response
Amplitude
Samples (n)


Chebyshev Type II LPF
Pole-Zero Plot
Imaginary Part
Real Part
Magnitude Response in dB
Magnitude (dB)
Frequency (Hz)
Phase Response
Phase (radians)
Frequency (Hz)
Group Delay
Group Delay (samples)
Frequency (Hz)
Impulse Response
Amplitude
Samples (n)



MSE & Percentage of Energy Lost:

```
The Mean Square Error is: 0.0025673
Power Loss Percentage for Butterworth LPF is: 3.81%

The Mean Square Error is: 0.0025886
Power Loss Percentage for Chebyshev Type I LPF is: 16.10%

The Mean Square Error is: 0.0015984
Power Loss Percentage for Chebyshev Type II LPF is: 3.57%

The Mean Square Error is: 0.0016746
Power Loss Percentage for Elliptic LPF is: 14.07%
```

The **Chebyshev Type II** filter (the third entry) is the best choice because it achieves both the minimum Mean Square Error and the minimum percentage of lost power.

TASK 3: Frequency Transformation using Pole-Zero Pattern Rotation

In this task, frequency transformations are performed on the designed Butterworth low-pass filter. The pole-zero pattern is rotated by πradians to convert the low-pass filter into a high-pass filter. A second transformation rotates the pattern to center the passband around $\pi/2$, producing a band-pass filter. Care is taken to ensure that the resulting filter coefficients remain real-valued.

For each transformed filter, the magnitude and phase response, pole-zero diagram, impulse response, and group delay are generated. This task demonstrates how frequency shifting in the z-plane affects filter characteristics and provides insight into designing filters through geometric manipulation of poles and zeros.

Code:

```
close all ;
clear all ;
clc ;

[x, Fs] = audioread('sample_audio_file.wav');
x = x(:,1);
Fp = 3000;
Fs_stop = 4000;
Ap = 1;
As = 50;
Wp = Fp / (Fs/2);
Ws = Fs_stop / (Fs/2);

%% Butterworth
[Nb, WnB] = buttord(Wp, Ws, Ap, As);
[bB, aB] = butter(Nb, WnB, 'low');
plot_filter_analysis(bB , aB , Fs, 0, 0);
sgtitle('Butterworth LPF');

[B_HPF, A_HPF]= LPF_transform_HPF(bB,aB);
plot_filter_analysis(B_HPF , A_HPF , Fs, 0, 0);
sgtitle('Butterworth HPF');


[B_BPF, A_BPF] = LPF_transform_BPF(bB, aB);
plot_filter_analysis(B_BPF , A_BPF , Fs, 0, 0);
sgtitle('Butterworth BPF');
```

LPF Transform HPF Function:

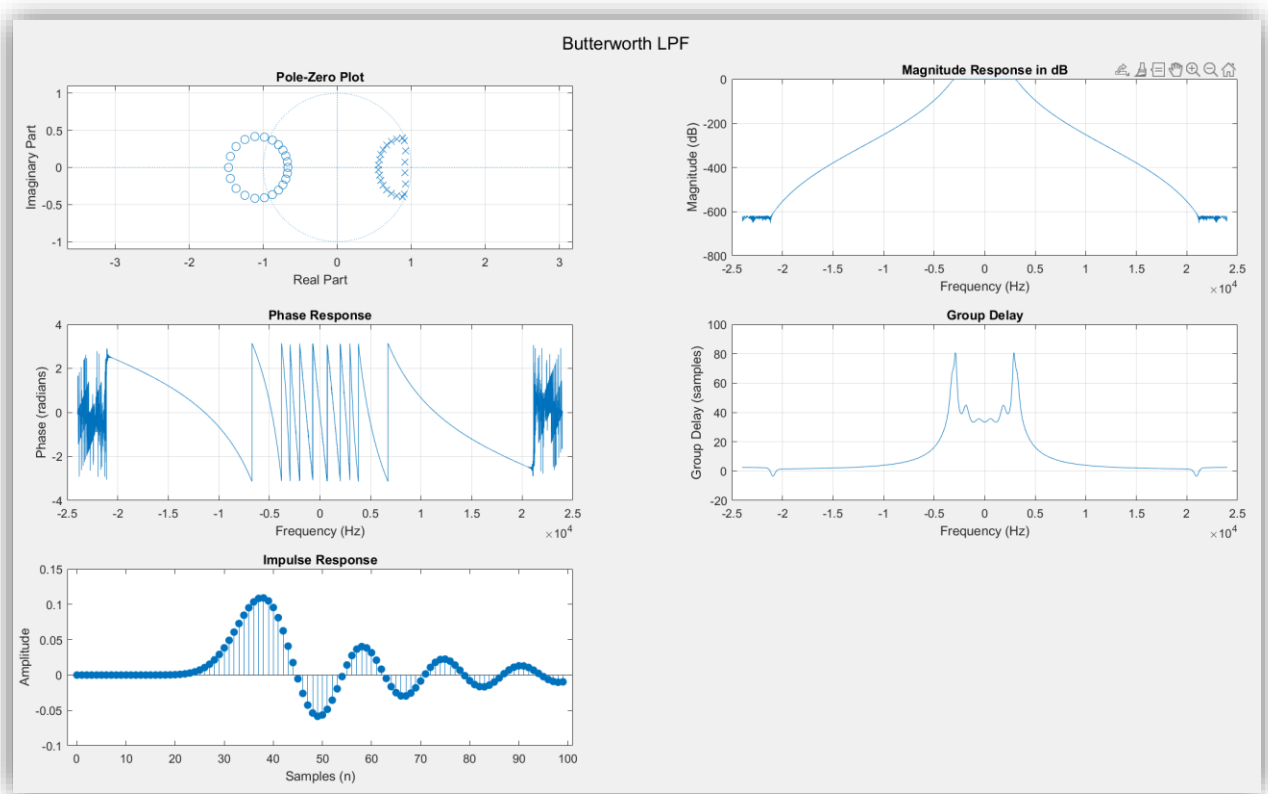
```
% input  : vectors b (LPF/HPF numerator coeff.)
%          a (LPF/HPF denominator coeff.)
% return : vectors B (HPF/LPF numerator coeff.)
%          A (HPF/LPF denominator coeff.)
function [B, A]= LPF_transform_HPF(b,a)
    n_b = length(b);
    n_a = length(a);
    % we want to replace each z with -z but due that z terms are organized
    % as even power, odd, even, odd , ... so z -> -z will result in
    % +, -, +, -, ... so we want to multiply be this sign sequence
    sign_mask_b = (-1).^(0:n_b-1);
    sign_mask_a= (-1).^(0:n_a-1);

    B = b .* sign_mask_b;
    A = a .* sign_mask_a;

end
```

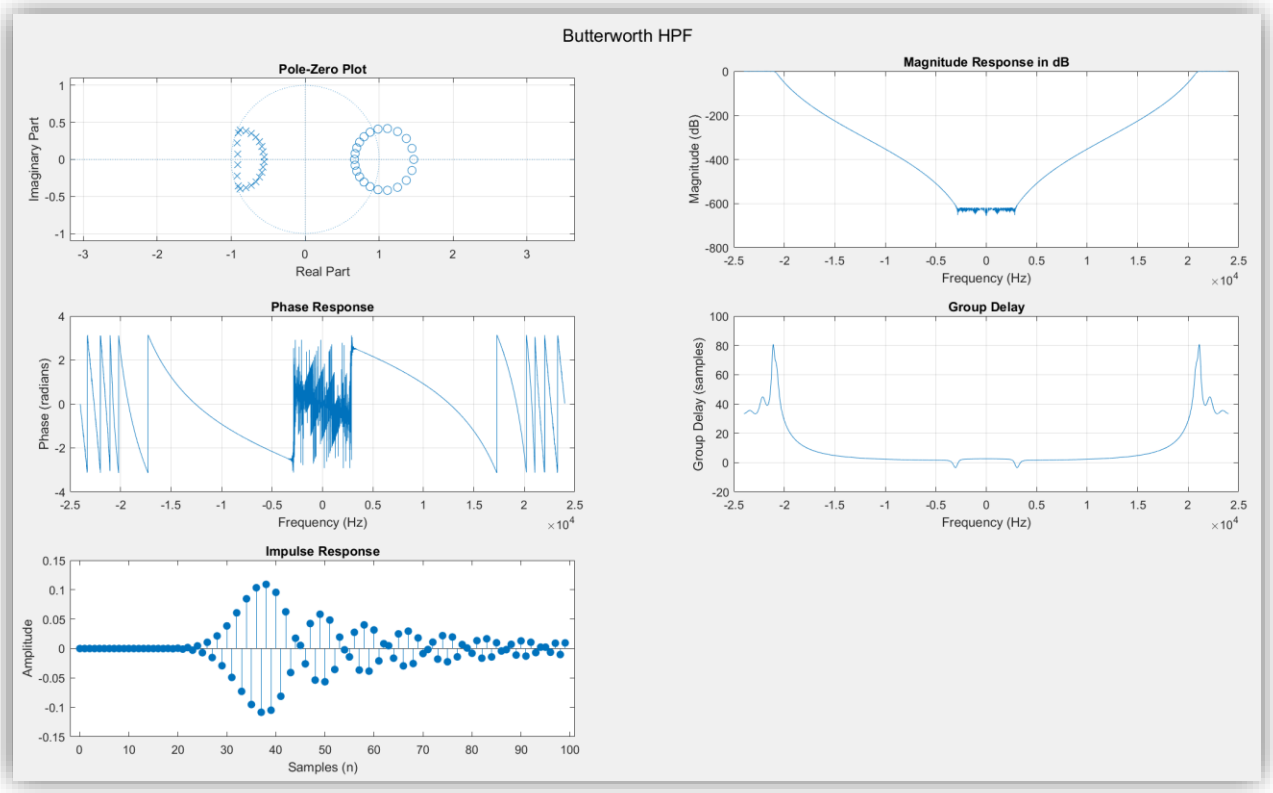
LPF Transform BPF Function:

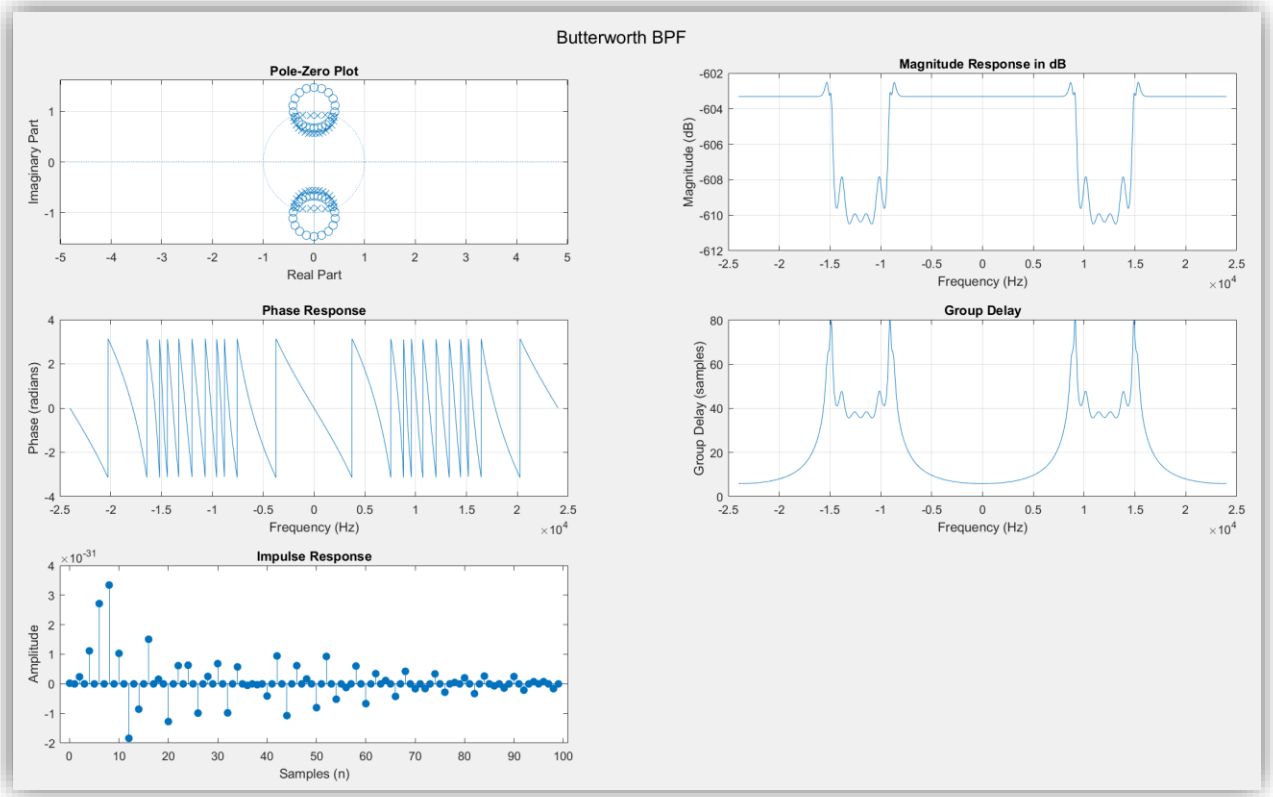
```
% input  : vectors b (LPF numerator coeff.)

%          a (LPF denominator coeff.)
% return : vectors B (BPF numerator coeff.)
%          A (BPF denominator coeff.)

function [B, A] = LPF_transform_BPF(b, a)
% Extract zeros and poles from LPF transfer function
    zeros_lpf = roots(b);
    poles_lpf = roots(a);
    k_lpf = b(1) / a(1);

    %To get a BPF centered at π/2 with real coefficients, we rotate the
    % pole-zero pattern by both +π/2 and -π/2, but each may have complex
coeff.
    % so we multiply both This creates conjugate pairs that result in
    % real coefficients

    % First transformation: rotate by +π/2 (multiply by j)
    zeros_1 = zeros_lpf * 1j;
    poles_1 = poles_lpf * 1j;

% Second transformation: rotate by -π/2 (multiply by -j)
    zeros_2 = zeros_lpf * (-1j);
    poles_2 = poles_lpf * (-1j);

% Combine both transformations (cascade the two systems)
    all_zeros = [zeros_1; zeros_2];
    all_poles = [poles_1; poles_2];
```

```
% Convert poles and zeros back to polynomial coefficients
    B = poly(all_zeros);
    A = poly(all_poles);

% Adjust gain (squared since we cascaded two systems)
    k_bpf = k_lpf^2;
    B = k_bpf * B;

% Normalize so that A(1) = 1
    B = B / A(1);
    A = A / A(1);

end
```

Plot:




Butterworth HPF
Pole-Zero Plot
Imaginary Part
Real Part
Magnitude Response in dB
Magnitude (dB)
Frequency (Hz)
Phase Response
Phase (radians)
Frequency (Hz)
Group Delay
Group Delay (samples)
Frequency (Hz)
Impulse Response
Amplitude
Samples (n)


Butterworth BPF
Pole-Zero Plot
Imaginary Part
Real Part
Magnitude Response in dB
Magnitude (dB)
Frequency (Hz)
Phase Response
Phase (radians)
Frequency (Hz)
Group Delay
Group Delay (samples)
Frequency (Hz)
Impulse Response
Amplitude
Samples (n)